

I2C Simulated Transport — Conversion Plan

Status: New document, companion to `roadmap.md`. Fully specifies `roadmap.md` tasks **1.6** (design), **1.7** (sim implementation), and the transport-facing half of **1.8** (CSP-layer integration) as a step-by-step, followable checklist — same house style as `roadmap.md`: numbered tasks, small (~1–3h), each with an explicit “done when.”

Why this document exists: `platform/{sim,real}/drivers/comms_i2c.c` are file-renamed already (`roadmap.md` 1.1–1.3) but not yet redesigned — the sim backend today is still a point-to-point Unix socket, one master and one slave, exactly like the old SPI model it replaced. Real I2C is a **shared bus**: every device physically sees every transaction on the same two wires, each device has an address, and only the addressed device responds. This plan closes that gap. The overriding priority, per direction: **the sim’s shape should be the same shape the real I2C driver will need**, so Phase 6’s hardware bring-up is a driver swap, not another redesign — and the code should stay abstracted enough that adding EPS/Thermals/Camera/Comms as real nodes later doesn’t require bespoke plumbing per board.

Part 1 — Real I2C, just enough to design against

I2C uses two shared wires (SDA = data, SCL = clock) that every device on the bus is physically connected to:

1. The master drives a **START condition**, then clocks out one address byte: 7 bits of device address + 1 read/write bit.
2. Every device on the bus sees this byte (it’s electrically shared). Only the device whose address matches pulls the line low to **ACK**; everyone else stays silent.
3. Data bytes flow (direction per the R/W bit), each ACKed by the receiver.
4. The master ends the transaction with a **STOP condition**.

The two properties that matter most for this design: **(a) addressing happens on the wire, before any payload**, and **(b) every device sees every transaction and self-filters** — the bus itself does no routing; the devices do.

Part 2 — Architecture decisions

2.1 Topology: hub-and-spoke socket standing in for the shared wire

	Today (point-to-point)	This plan (shared bus)
Connections	One socket per OBC↔board pair	One well-known socket; every board connects to it
OBC's role	accept()s exactly one connection	accept()s in a loop, holds N connections open
Delivery	Direct — there's only one other party	OBC relays (broadcasts) every received frame to all other connected boards

OBC hosts the bus because it already plays this role today (it binds/listens/accepts in the current sim), it's already the CSP network hub (address 1, `is_default=1`), and it's the real project's actual I2C bus controller too (Zynq PS is the I2C master in the real topology) — no new asymmetry is being invented, just extended to more than one peer.

2.2 Wire frame format

Keep the existing length-prefix framing (it already avoids stream ambiguity and is proven by `comms_bus_test`) — just prepend two address bytes, directly mirroring I2C's own “address before payload” shape:

dest_addr	src_addr	length(u16)	payload (N bytes)
1 byte	1 byte	2 bytes	

`dest_addr/src_addr` reuse the existing CSP address table (`roadmap.md`): OBC=1, ADCS=2, EPS=3, THERMALS=4, CAMERA=5, COMMS=6. No new addressing scheme — the physical layer just starts carrying the same addresses CSP already assigns.

2.3 Addressing & routing — broadcast, then filter

Every connected node receives every frame (this is what makes it a faithful shared-bus model, not just a fancier point-to-point link). Each node's receive path checks `dest_addr` against its own address:

- **Match** → deliver the payload to the caller.
- **No match** → discard silently, exactly like a real I2C device that never ACKs an address that isn't its own.

2.4 Master/slave roles — what's kept, what's simplified

Real I2C only lets the master initiate a transaction; a slave never sends unsolicited. This project's CSP traffic is bidirectional by design (subsystems push telemetry proactively, not

just in response to a poll) — keep that duplex behavior rather than forcing strict master-initiated-only semantics. This is a deliberate, named simplification: electrical fidelity isn't the goal here, matching `roadmap.md`'s own guiding principle — “we didn't make a motor spin, we made the logic flow.”

2.5 The one necessary breaking change: `send/receive` need an address

Today: `send(data, length)` has an implicit destination, because point-to-point meant there was only ever one other party. A shared bus needs an explicit one. Extend `CommsBus_t`:

```
typedef struct {
    CommsBusStatus_t (*initialize)(uint8_t my_address, int is_master);
    int (*send)(uint8_t dest_addr, const uint8_t *data, uint16_t length);
    int (*receive)(uint8_t *src_addr_out, uint8_t *buffer, uint16_t max_length);
} CommsBus_t;
```

This isn't scope creep — it's the change that makes the sim and the eventual real driver **shaped alike**: a real I2C HAL call also takes a device address (e.g. `HAL_I2C_Master_Transmit(&hi2c, dev_address, ...)`). Designing the sim's contract around an address parameter now means Phase 6's real backend fills in the same function signatures with real HAL calls instead of socket code — a body swap, not an interface redesign.

2.6 Where node identity lives — keep `comms_bus.h` generic

`comms_bus.h`'s job stays “the medium-agnostic bus contract” — it shouldn't hardcode CSP's address table or know anything about which board is which. `my_address` is just a `uint8_t` parameter the caller (`csp_if_i2c.c`, later) fills in from the CSP address it's already configured with (`csp_network_init(my_address, is_master)` already receives this value today — it just needs to flow one layer further down). Real hardware needs this exact value too, to program the I2C peripheral's own slave-address register — so it's honestly medium-agnostic, not a sim-only concept.

Part 3 — Step-by-step conversion checklist

#	Task	Depends on	Est.	Done when
I2C.1	Write Part 2 above as the header comment atop both comms_i2c.c files (satisfies roadmap.md 1.6 exactly)	roadmap.md 1.3 done	1h	Comment matches Part 2, no code changes yet
I2C.2	Extend shared/interfaces/comms_bus.h: add dest_addr to send, src_addr_out to receive, my_address to initialize	I2C.1	1h	Header compiles standalone; CommsBus_t shape matches 2.5
I2C.3	Implement the hub/broadcast loop in platform/sim/drivers/comms_i2c.c's master path — accept() in a loop instead of once, fan out each received frame to every other connected client	I2C.2	3h	Two trivial client connections can both connect to one running master process at the same time
I2C.4	Implement client-side framing: send prepends dest_addr/src_addr; receive reads the address	I2C.3	2h	A 3-process test (1 master + 2 slaves) shows a message addressed to slave B being silently discarded by the slave
I2C.5	Implement full suite of bus master and slave (non-matching addresses) before receiving a frame for address X or	I2C.3 5	2.5h	Positional argument test still passes the address check against correct manual run shows correct

~11.5 hours total — comparable to roadmap.md's own estimate for tasks 1.6–1.8 combined (8h), with the difference spent making the addressing/filtering behavior explicit and tested rather than assumed.

Part 4 — How this sets up the real I2C driver later

platform/real/drivers/comms_i2c.c's future real implementation (roadmap.md Phase 6) receives the exact same (dest_addr, data, length) / (src_addr_out, buffer, max_length) shape defined in 2.5. Swapping the sim's broadcast-and-filter socket logic for real HAL_I2C_Master_Transmit()/HAL_I2C_Slave_Receive() calls is then a **body-only change** — the function signatures, and everything above this layer (csp_if_i2c.c, csp_network.c, CSP itself), never need to change again for this reason.

Part 5 — Definition of Done

Same bar as roadmap.md's Foundation DoD, applied to this specific piece:

- [?] Documented (this file) and cross-referenced from roadmap.md
 - [?] Has a working, minimal example — the 3-node addressing test from I2C.6, not just a header
 - [?] Used by a real consumer before being called done — csp_network_init() for both OBC and ADCS, exercised by position_command_test
 - [?] roadmap.md 1.6's own done-when is met (design doc, no code, atop comms_i2c.c) before I2C.2 starts writing code
 - [?] roadmap.md 1.7's own done-when is met (compiles, a trivial two-process send/receive test passes) by I2C.4
 - [?] roadmap.md 1.8's transport-facing half (CSP calls the new signature correctly) is met by I2C.5
-

I2C Simulated Transport — Conversion Plan. Companion to roadmap.md; supersedes the placeholder note in roadmap.md 1.6 once that task is marked done against this document.